CS633: Parallel Computing

Assignment–1

Report

Group Number 61

Aayush Kumar (230027)
Aritra Ambudh Dutta (230191)
Cezan Vispi Damania (230310)
Suvid Goel (231065)
Swapnil Garg (231069)

Date: February 14, 2026

1 Code Description

Our implementation simulates a parallel data processing pipeline using strictly *point-to-point MPI communication*. The system operates on a linear topology where every process functions in a dual capacity: as a **sender**, it dispatches *rank-dependent pseudo-random data* to downstream neighbors at offsets **D1** and **D2**; as a **receiver**, it processes incoming data using specific mathematical transformations.

For **T** iterations, senders transmit arrays of M doubles to their targets. The receiving ranks perform element-wise computations—squaring the values for the $D1$ channel and computing natural logarithms for the $D2$ channel—before returning the results. Upon receipt, the sender updates its buffers using modulo and multiplication rules to prepare for the next cycle.

The simulation concludes with a **binary-tree reduction** to identify the global maximums. This algorithm efficiently gathers the final results at Rank 0 in $O(\log P)$ steps, strictly adhering to the assignment constraint that forbids `MPI_Reduce` for data values.

1.1 Modular Decomposition

The program execution is divided into three main phases: Initial Setup, The Iterative Core, and Final Aggregation.

Phase 1: Initialization and Setup (One-time)

- **Initialization & Allocation:** The program begins by parsing command-line arguments ($M, D1, D2, T, seed$) and initializing the MPI environment. Memory buffers are allocated conditionally to strictly adhere to the topology and minimize overhead:
  - `buf_d1` and `buf_d2` (outgoing buffers) are allocated only if `rank + D1` is a valid receiver, i.e. `rank + D1 < P`.
  - `data_d1` and `data_d2` (incoming buffers) are allocated only if `rank - D1` is a valid sender, i.e. `rank - D1 >= 0`.
- **Data Generation:** The random number generator is seeded using `srand(seed)`. The primary send buffer (`buf_d1`) is populated using the specified formula:

$$\texttt{buf\_d1}[i] = (\texttt{double})\texttt{rand}() \times (\texttt{rank} + 1)/10000.0$$

  To ensure consistency as required by the problem statement, if a rank is also a valid D2-sender, the contents of `buf_d1` are explicitly copied to `buf_d2`.

Phase 2: The Iterative Core (T iterations)
Inside the main loop, each iteration executes four logical rounds.

- **Parallel ODD/EVEN Communication in Block of Size 'D' (Round1 & Round3):** In this topology, every rank R (except boundaries) acts simultaneously as a sender to $R + D$ and a receiver from $R - D$. Since blocking MPI calls are used, a uniform "Send then Recv" order would cause a deadlock resulting in serial communication form last rank to first rank in blocks of size D(first for d1, then for d2).

  We employ a **parity-based odd-even scheduling** strategy to interleave these operations. Ranks are divided into blocks of size D, and the execution order is determined by:

$$\texttt{parity} = \left\lfloor \frac{\texttt{rank}}{D} \right\rfloor \pmod 2$$

  - **Round 1 (Forward Data Transfer):** Every rank attempts to send data forward and receive data from the back.
    * **Even Blocks: Send** to $R + D$, *then* **Recv** from $R - D$.
    * **Odd Blocks: Recv** from $R - D$, *then* **Send** to $R + D$.
  - **Round 3 (Return Results):** The data flow is reversed (ranks return computed results to the left). To maintain safety, the priority is flipped:
    * **Odd Blocks: Send** result to $R - D$, *then* **Recv** result from $R + D$.
    * **Even Blocks: Recv** result from $R + D$, *then* **Send** result to $R - D$.

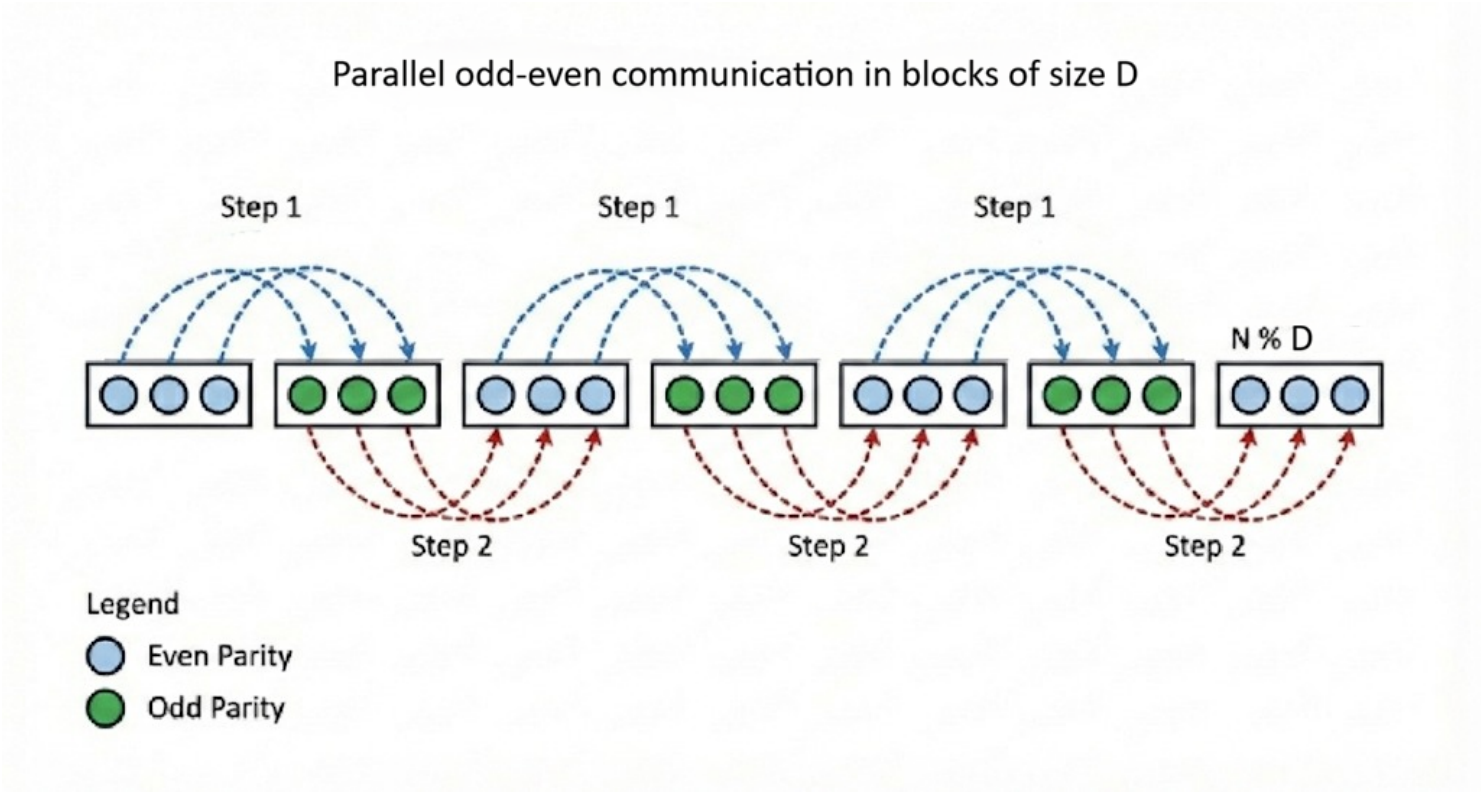

Figure 1: Parallel odd-even communication (R1) scheme illustration, where adjacent process blocks exchange data in two ordered phases - Step 1 (forward) and Step 2 (reverse) using one-to-one corresponding node communication to eliminate cyclic dependencies and prevent deadlock.

Explanation: Grouping by rank $= \lfloor \texttt{rank}/D \rfloor$ divides the processes into contiguous blocks of size D. Adjacent blocks always have opposite parity (e.g., for $D = 2$, the pattern is $0, 0, 1, 1, 0, 0 \ldots$).

Any communication pair $(rank, rank + D)$ always spans two adjacent blocks. Since these blocks have opposite parity (0 vs 1), our conditional logic ensures that one side enters the `Send`-first branch while the other enters the `Recv`-first branch.

Specifically:

- In **Round 1**, the "Even" block (the data source) sends first, matching the "Odd" block which receives first.
- In **Round 3**, the roles are reversed: the "Odd" block (now holding the computed result) sends first, matching the "Even" block which receives first.

This perfect complementarity guarantees that no circular wait can occur.

This scheme effectively partitions the processes into two sets of approximately equal size ($P/2$ ranks in the even group, $P/2$ in the odd group). Consequently, for any single communication phase (R1 or R3) and distance D, all required point-to-point exchanges are completed in just **two** logical steps. This yields a **parallelism factor of ≈ 0.5 per phase**, meaning that at any given moment, half of the active processes are sending data simultaneously while the other half are receiving.

- **Computation (R2) and Update (R4):**
  - **Computation (R2):** After successful reception, valid receiver ranks invoke the dedicated routines `compute_d1()` and `compute_d2()`. The function `compute_d1()` performs element-wise squaring on `data_d1`, while `compute_d2()` applies the natural logarithm to `data_d2`. These processed results are then returned to the corresponding senders in Round 3.
  - **Senders (R4):** After receiving the processed results in Round 3, senders update their buffers using the update_buf_d1 function (modulo 100000 for senders receiving data from rank + d1) or update_buf_d2 function (for senders receiving from rank + d2).
- **Boundary Handling & Memory Optimization:** All communication calls are guarded by boundary checks (e.g., `rank + D < P`). To strictly minimize memory footprint, we conditionally allocated buffers; specifically, `buf_d` and `data_d` buffers were only allocated if the corresponding neighbour exists. If a neighbour is out of range, the specific Send or Recv call is skipped, but the process still participates in the ordering logic for the valid side of the connection.

Phase 3: Final Aggregation (One-time)
Upon completion of the T iterations, the program transitions to the result aggregation phase. Since the assignment specification prohibits the use of `MPI_Reduce` for data values, we implemented a manual **binary-tree reduction strategy** to consolidate global maximums efficiently.

- **Local Maximum Extraction:** Before communication begins, every valid sender scans its final resident buffers (`buf_d1` and `buf_d2`) to identify the local maximum values (Max_{local}). This ensures that subsequent communication only transmits less data (2 doubles per rank) rather than full arrays, minimizing communication latency.
- **Manual Tree Reduction:** We implemented a binary reduction algorithm that aggregates results in $\boxed{O(\log P)}$ steps.
  - **Logic:** The reduction proceeds in steps where the stride (`diff`) doubles each iteration $(1, 2, 4, \dots)$.
  - **Role Assignment:** In step k with stride S, a rank determines its role using bitwise modulo logic:
    - **Senders:** Ranks satisfying `rank % (2S) == S` send their current maximums to `rank - S`.
    - **Receivers:** Ranks satisfying `rank % (2S) == 0` receive data from `rank + S`, compare it with their current values ($\max(M_{local}, M_{received})$), and carry the new maximum to the next step.
  - **Result:** This structure forms a reverse binary tree where partial results flow leftward, eventually consolidating the global maximums for D1 and D2 at Rank 0.

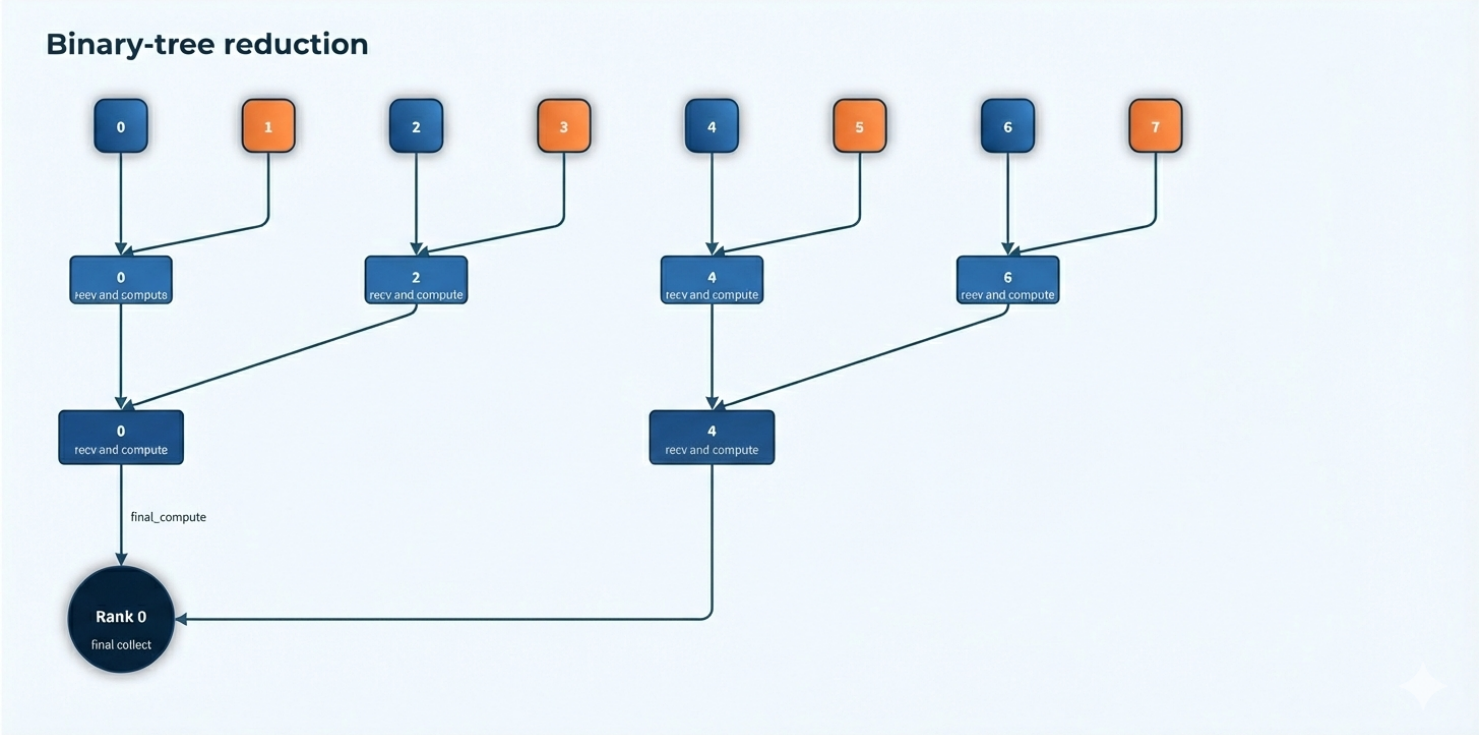

Figure 2: Binary-tree (recursive doubling) reduction pattern used for manual aggregation of local maximum values. Results propagate toward Rank 0 in $O(\log P)$ steps.

- **Global Timing & Reporting:** Finally, the total execution time is aggregated using `MPI_Reduce` (with `MPI_Op` as `MPI_MAX`) for calculating Maximum Time among all processes as permitted. Rank 0 then writes the global D1 maximum, D2 maximum, and the elapsed wall-clock time to the output file in the requested format.

1.2 Code Snippets

The following snippets highlight the key logic components of our implementation.

1. Parallel ODD/EVEN Communication in Block of Size 'D' (Communication Round 1)

```
// Parity check to determine Send/Recv order to prevent deadlock
// We check if the rank belongs to an "Odd" or "Even" block of size D1
if ( !((rank / d1) & 1) ) {
    // Even Block: Send First
    if (rank + d1 < p)
        MPI_Send(buf_d1, m, MPI_DOUBLE, rank + d1, 0, COMM);
    if (rank - d1 >= 0)
        MPI_Recv(data_d1, m, MPI_DOUBLE, rank - d1, 0, COMM, &status);
} else {
    // Odd Block: Recv First
    if (rank - d1 >= 0)
        MPI_Recv(data_d1, m, MPI_DOUBLE, rank - d1, 0, COMM, &status);
    if (rank + d1 < p)
        MPI_Send(buf_d1, m, MPI_DOUBLE, rank + d1, 0, COMM);
}
```

2. Manual Binary Tree Reduction (Aggregation Phase)

```
// Custom reduction since MPI_Reduce is forbidden for data
// Aggregates local maximums in O(log P) steps
int diff = 1;
int num = p - d1;
while(diff < num){
```

```
    // Determine role: Sender or Receiver based on current stride (diff)
    // Using bitwise logic: (1 << diff) represents the current block size mask
    if((rank % (diff<<1)) == diff){
        // Sender: Pass max to neighbor 'diff' steps behind
        if(rank - diff >= 0)
            MPI_Send(a, 2, MPI_DOUBLE, rank-diff, 0, COMM);
    }
    else if(((rank % (diff<<1)) == 0) && (rank + diff < num)){
        // Receiver: Get data, compare with local max, and continue
        MPI_Recv(b, 2, MPI_DOUBLE, rank + diff, 0, COMM, &status);
        if(a[0] < b[0]) a[0] = b[0]; // Update Max D1
        if(a[1] < b[1]) a[1] = b[1]; // Update Max D2
    }
    diff <<= 1; // Double the stride (Recursive Doubling)
}
```

2 Code Compilation and Execution

2.1 Compilation

To compile the code, we use the standard MPICC wrapper, linking the math library for logarithmic operations:

```
mpicc -o src src.c -lm
```

2.2 Execution Strategy

The program accepts arguments in the order: $M, D1, D2, T, Seed$, and generates and output file `out_<p>.txt`, where p is the number of processes in the argument.

```
# Manual syntax: mpirun -np <P> ./src <M> <D1> <D2> <T> <SEED>
mpirun -np 16 ./src 262144 2 4 10 1000
```

2.3 Automated Batch Submission (SLURM)

For the performance analysis, we utilized SLURM batch scripts to automate execution across the three required process counts ($P \in \{8, 16, 32\}$) and two data sizes ($M \in \{262144, 1048576\}$).

The scripts enforce the constraint of **16 processes per node** and automatically repeat each experiment 5 times to generate statistical data for the boxplots.

Below is the representative submission script for the $P = 32$ case (using 2 nodes):

```
#!/bin/bash
#SBATCH --job-name=p_32
#SBATCH -N 2                    # Request 2 nodes for 32 processes
#SBATCH --ntasks-per-node=16    # Enforce 16 tasks per node
#SBATCH --cpus-per-task=1
#SBATCH --time=00:15:00
#SBATCH --partition=cpu
#SBATCH --output=results_p32_%j.log
#SBATCH --error=results_p32_%j.log

module load compiler/oneapi-2024/mpi

# Configuration Constants
P=32
D1=2
D2=4
T=10
SEED=1000

```

```
# Nested loops: 5 repetitions x 2 Data Sizes
for run in {1..5}; do
    for M in 262144 1048576; do
        mpirun -np $P ./src $M $D1 $D2 $T $SEED
    done
done
```

Figure 1: SLURM Job Script (P=32)

Similar scripts were used for $P = 8$ and $P = 16$, with `#SBATCH -N 1` and the corresponding `P` value adjusted.

To execute the job on the Paramrudra cluster, we submit the SLURM script using:

```
sbatch job_script.sh
```

Once submitted, SLURM assigns a job ID and generates a log file of the form `results_<p>_<jobid>.log` along with an output file of the form `out_<p>.txt`(here p is the number of processes). The log file contains job details of the SLURM run while the output file provides a structured output for each run. Example (out_32.txt):

```
99936.000000 1416360.081586 0.682352
99936.000000 1416360.081586 1.029124
99936.000000 1416360.081586 0.736486
99936.000000 1416360.081586 1.002146
99936.000000 1416360.081586 0.713446
99936.000000 1416360.081586 1.066106
99936.000000 1416360.081586 0.733719
99936.000000 1416360.081586 1.062412
99936.000000 1416360.081586 0.732949
99936.000000 1416360.081586 1.041779
```

Each output line contains the maximum of data_buf_d1, followed by the maximum of data_buf_d2 and then the total time taken (time for T iterations + final communication/computation). The output file contains 5 of these repetitions for each data size, resulting in 10 outputs, per process job script, which we extract and use to compute averages and generate the box plots presented in the Results section.

3 Results

We tested our implementation using $np = 8, 16, 32$ for two data sizes (262144 and 1048576 elements). Each configuration was executed multiple times, and the results are presented using a single box plot to show both the average execution time and its variability.

Observations

- For both data sizes, the **execution time consistently increases** as the number of processes increases from 8 to 32, indicating that increasing the number of processes does not improve performance in our implementation, due to **higher communications between processes** as the number of processes increases.
- The likely reason is that **communication overhead grows with the number of processes**, increasing communication cost, which **outweighs the benefit of the increased parallel computation** by using multiple processes.
- For the smaller data size (262144), the computation per process is relatively small, so **communication overhead dominates quickly**. Even for the larger data size (1048576), although more computation is involved, the excess, **communication cost still outweighs** the parallel computation speedup.
- The **box plots show low variance** across repeated runs, confirming that this trend is consistent and not due to random fluctuations.

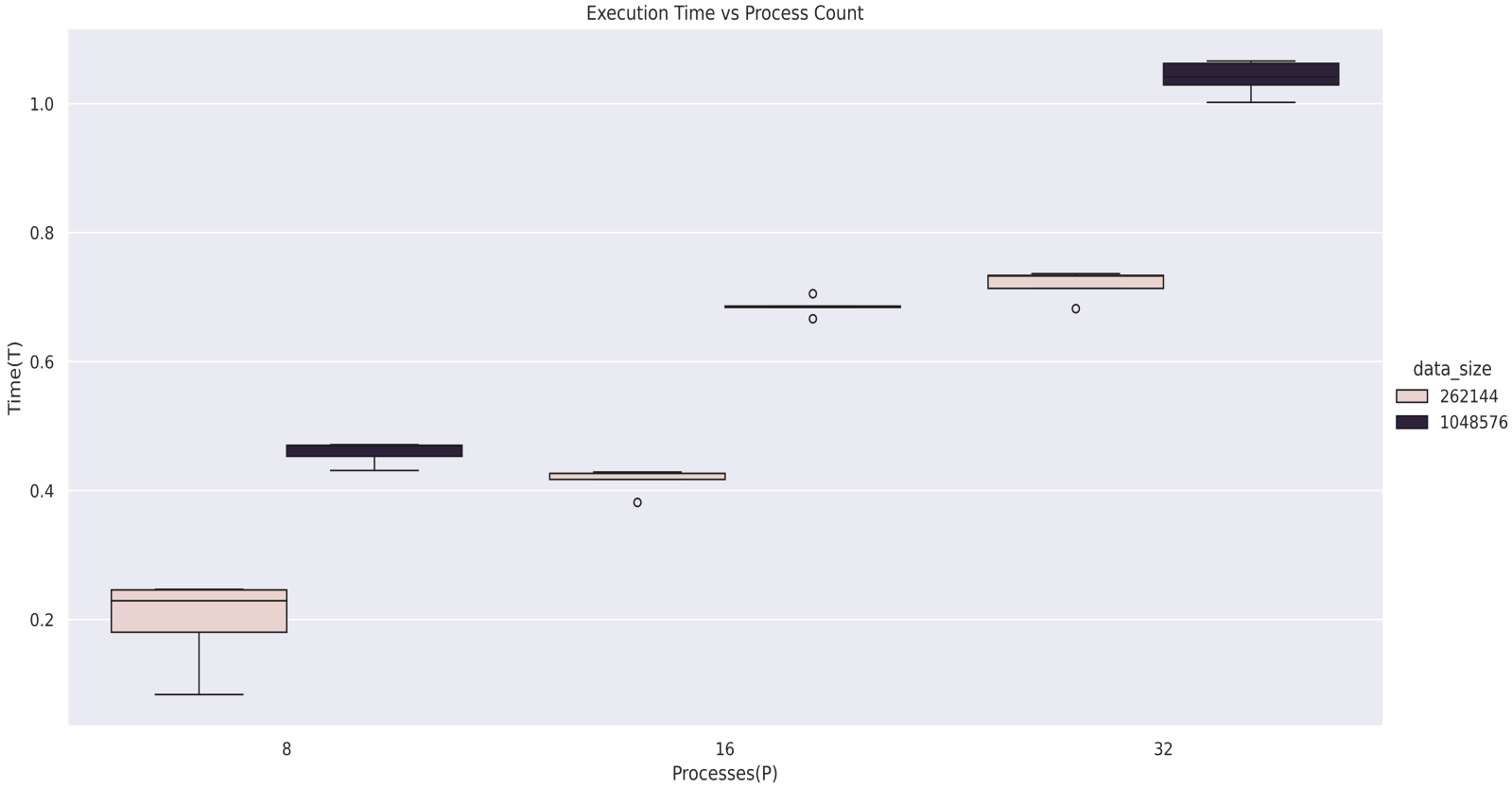

Figure 3: **Scalability Analysis:** Execution time vs. Number of Processes (P) for $M = 262144$ and $M = 1048576$. Boxplots represent distribution over 5 runs.

Overall, we observe that our implementation does not scale efficiently with increasing process count because the **overhead of inter-process communication becomes the dominant factor**, since the data size remains constant across different number of processes.

4 Conclusion

In this assignment, we implemented and evaluated an MPI-based parallel communication strategy under varying process counts and input sizes. The experiments were conducted on the Paramrudra cluster with multiple runs per configuration, and the results were analyzed using statistical visualization. The evaluation provides insight into the performance characteristics of the implementation.

- The program is designed strictly using **point-to-point MPI communication**, where each rank operates simultaneously as a sender and receiver within a linear topology using distance parameters D_1 and D_2.
- A **parity-based odd-even scheduling** mechanism was implemented to ensure deadlock-free execution while using blocking `MPI_Send`/`MPI_Recv`, guaranteeing complementary communication ordering between adjacent blocks, for faster **communication between processes**.
- A **binary-tree reduction** algorithm was developed to aggregate global maximum values in $O(\log P)$ steps, for efficient communication.
- Upon running, the trends in performance show that **execution time increases** as the number of processes increases from 8 to 16 to 32 for both data sizes.

- This behavior indicates that **communication overhead** become more significant at higher process counts. While **parallelization distributes the computational workload**, the associated **communication time** limits the achievable speedup for the tested problem sizes.
- The results highlight the importance of maintaining a balanced **computation-to-communication ratio** in parallel program design.
- Overall, the study demonstrates practical considerations in achieving **efficient and scalable** parallel execution.

5 Workload Distribution

Name	**Roll Number**	**Contribution**
Aayush Kumar	230027	20%
Aritra Ambudh Dutta	230191	20%
Cezan Vispi Damania	230310	20%
Suvid Goel	231065	20%
Swapnil Garg	231069	20%